2. Planning and Controlling the Work (Walk) 🚶

This section focuses on the "mechanics" of project management—moving from the "what" (strategy) to the "how" (execution). Here, you will learn to plan, structure, and control scope, time, and budget using repeatable processes that ensure delivery stability. The goal is to create a robust plan that can withstand the realities of daily project work.

2.1. Scope Management: Defining the Product

Scope management is the backbone of project planning. It involves defining, structuring, and strictly maintaining the boundaries of what the project will deliver. The primary objective is to ensure that the team builds exactly what is needed—no more, no less—thereby avoiding the common pitfall of "gold-plating" (adding unnecessary features) or "scope creep" (uncontrolled expansion).

2.1.1. Requirements Gathering Techniques

Before any backlog is created or code is written, the project team must collect and elaborate on all available requirements. This is a discovery process that involves multiple techniques to ensure no critical need is missed.

- **Facilitated Workshops:** Often referred to as Joint Application Development (JAD) sessions, these bring cross-functional stakeholders (e.g., developers, users, business owners) together. The goal is to build consensus on requirements and reconcile conflicting needs in real-time.
- **Prototypes and Mockups:** Creating visual representations allows users to "play" with the concept and provide feedback early, drastically reducing the risk of misunderstandings during development.
- **Documentation Analysis:** Reviewing existing SOPs, legal regulations, and legacy system logs to find requirements that are mandatory but might not be top-of-mind for stakeholders.
- **BA Techniques:** employing standard business analysis methods such as Interviewing, Surveys, and Job Shadowing to extract information from stakeholders.
- **Affinity Diagrams:** Organizing large amounts of data (ideas, requirements, sticky notes) into natural relationships or groupings.

2.1.2. BA Elaboration (Use Cases, User Story Mapping)

Once raw requirements are gathered, they must be structured into a coherent narrative that developers can implement. This process is often called "Elaboration".

- **User Story Mapping:** A powerful visualization technique that arranges user activities (the "backbone" of the user journey) in a horizontal flow and details (tasks/stories) vertically by priority. It prevents the "flat backlog" problem where a team creates a list of disconnected features. Instead, it forces the team to think in terms of user journeys and "slices" of value.
- **Use Case Modeling:** This describes how a system interacts with external users (actors) to achieve a specific goal. It is more formal than user stories and is excellent for defining complex system behaviors and edge cases.
- **Business Process Modeling (BPM):** Mapping out the current ("As-Is") and future ("To-Be") business workflows to identify efficiencies.
- **Entity Relationship Diagram (ERD):** Visualizing the data structure and relationships within the system to ensure the backend can support the requirements.
- **NFR Analysis:** Specifically gathering and defining Non-Functional Requirements like security, performance, and scalability.

2.1.3. Work Breakdown Structure (Epics → Stories)

The **Work Breakdown Structure (WBS)** is a hierarchical decomposition of the total scope of work to be carried out by the project team. It breaks large, complex deliverables into smaller, manageable sections.

- **Hierarchy:** The typical flow in modern agile/hybrid projects is **Epics** (large initiatives) → **Features** (and Enabler Features) → **Stories** (and Enabler Stories/Spikes) .
- **Criteria for Stories (INVEST):** To ensure a User Story is ready for the team, it should follow the INVEST model: **I**ndependent, **N**egotiable, **V**aluable, **E**stimable, **S**mall, and **T**estable.
- **SMART Tasks:** For non-development tasks, use the SMART criteria: **S**pecific, **M**easurable, **A**chievable, **R**elevant, and **T**ime-bound.

2.1.4. Definition of Ready and Done

To prevent churn and ensure quality, the team must agree on strict "Quality Gates" for work items. These are explicit agreements that prevent bad work from entering or leaving the sprint.

- **Definition of Ready (DoR):** The set of attributes defining actionable stories. A story cannot be pulled into a sprint unless it meets the DoR (e.g., has acceptance criteria, is estimated, dependencies identified).
- **Definition of Done (DoD):** An agreed-upon checklist that must be completed before an item is considered finished.
    - **Story Level:** Code is peer-reviewed, unit tests are passed, documentation is updated, and it is deployed to the test environment.
    - **Sprint/Team Increment:** All stories are accepted, all tests passed, NFRs were met, no major bugs, and the update is deployed to a specific environment.
    - **Release/System Increment:** Features meet all acceptance criteria, the increment is accepted, and regression testing is completed.
    - **Project/Solution Increment:** All above criteria are met, plus all deliverables are accepted.

2.1.5. Scope Baselining and Change Management

The **Scope Baseline** is the approved version of the scope statement, WBS, and WBS dictionary. Once baselined, it can only be changed through formal change control procedures.

- **Scope Creep:** The uncontrolled expansion to product or project scope without adjustments to time, cost, and resources.
- **Change Management (Triple E Framework):** To manage changes effectively without stifling necessary adaptations:
    - **Enable:** Establish procedures that enable change management (e.g., standard Change Request forms, defined workflows, a Change Log).
    - **Empower:** Give authority and resources to the Change Control Board (CCB) or the PM to make decisions quickly.
    - **Encourage:** Align changes with KPIs and ensure the process is followed by the team and stakeholders.
- **Traceability:** A **Requirements Traceability Matrix (RTM)** links requirements from their origin to the deliverables that satisfy them and the test cases that verify them.

2.2. Estimation and Cost Management

Mastering project finance and estimation is critical. A project that delivers on time but runs out of money is a failure. You must learn to predict costs and track them against a baseline.

2.2.1. Estimation Models

- **Bottom-up:** This involves estimating individual components (at the task or story level) and rolling them up to a total. It is the most accurate method because it is granular, but it is also the most time-consuming.
- **Top-down (Analogous):** Using historical data from similar past projects to estimate the current one. It is fast but less accurate, best used in early phases.
- **Parametric:** Using statistical relationships between historical data and other variables (e.g., COCOMO II, SLIM) to calculate estimates.
- **3-Point Estimate:** To account for uncertainty, this method uses three estimates: Optimistic, Pessimistic, and Most Likely. It can be calculated as a simple average or a weighted average (PERT).

2.2.2. Techniques: Expert Judgment, Delphi, Planning Poker

- **Expert Judgment:** Leveraging the knowledge of Subject Matter Experts (SMEs) or experienced team members.
- **Delphi Method:** A structured communication technique where experts answer questionnaires in two or more rounds to reach consensus without bias.
- **Planning Poker:** An agile consensus-based technique using cards (often Fibonacci) to estimate effort. It avoids the "anchoring" bias by having team members reveal estimates simultaneously.

2.2.3. Budget Calculation and Cost Baseline

The budget is not just the cost of doing the work; it includes reserves, management fees, and overheads. Follow these 9 steps to calculate the project budget :

1. **Gather Requirements:** Understand exactly what needs to be built.
2. **Define Scope (WBS):** Break the work down into estimable chunks.
3. **Estimate the Scope:** Apply one of the estimation models above.
4. **Identify the Team:** Build the staffing plan.
5. **Build the Timeline:** Create the schedule with reserves and buffers.
6. **Agree Rates:** Confirm billing rates with the account manager.

7. **Get Costs:** Obtain average total costs for project roles from Operations.
8. **Calculate PNL:** Calculate Project Profit and Loss using the timeline, ramp-up plans, rates, and costs.
9. **Final Agreement:** Agree on the final budget to be proposed to the client.

2.3. Schedule Management and Time Control

2.3.1. Gantt Charts, Network Diagrams

- **Gantt Charts:** The most common tool for project scheduling. They visualize the project schedule, showing tasks, durations, start/end dates, and dependencies. Tools include MS Project, ClickUp, Miro, and Excel .

+1

- **Network Diagrams:** Visual representations of project tasks and their logical flow (dependencies), used to identify the critical path.

2.3.2. Critical Path and Critical Chain

- **Critical Path Method (CPM):** Identifies the longest path through the schedule with zero or negative total float. Tasks on this path dictate the shortest possible project duration.
- **Critical Chain Project Management (CCPM):** A technique that modifies the schedule to account for limited resources. It focuses on managing buffers (Project Buffer, Feeding Buffer) rather than just task deadlines to protect the delivery date.

2.3.3. Schedule Compression: Fast Tracking, Crashing

When a project is falling behind schedule, the PM can use specific techniques to recover time. These should be applied carefully as they involve trade-offs :

- **Cutting the Scope:** Reducing the deliverables to meet the deadline.
- **Cutting the Quality:** Reducing testing or review cycles (highly risky).
- **Fast Tracking:** Performing activities in parallel that were originally planned in sequence (increases risk of rework).
- **Crashing:** Adding more resources (people, money) to critical path tasks (increases cost).
- **Replacing Team Members:** Swapping underperforming or junior members with more experienced staff who can work faster.

- **Cutting Vacations/Day-offs:** Asking the team to delay planned time off.
- **Overtime Work:** Increasing working hours (e.g., weekends) to catch up.

2.3.4. Rolling-wave Planning

Rolling-wave planning is an iterative planning technique essential for projects with high uncertainty. Instead of planning the entire project in detail from day one, the work to be accomplished in the near term is planned in detail (low level), while work in the future is planned at a higher level. As the project progresses and the "wave" moves forward, the high-level tasks are broken down into granular details.

2.4. Risk Management Fundamentals

Risk management is the proactive process of identifying, analyzing, and responding to risk factors throughout the life of a project. It is not a one-time activity but a continuous cycle.

2.4.1. Risks vs. Opportunities vs. Issues

- **Risk:** An uncertain event that, if it occurs, has a positive or negative effect on project objectives.
- **Opportunity:** A positive risk that can be exploited to benefit the project.
- **Issue:** A risk that has already happened; a current problem needing immediate resolution.
- **Assumption:** A factor considered true without proof. If an assumption proves false, it becomes a risk.

2.4.2. Qualitative & Quantitative Analysis

- **Qualitative Analysis:** Prioritizing risks by assessing their **Probability** and **Impact** (e.g., High/Medium/Low matrix).
- **Quantitative Analysis:** Numerically analyzing the effect of identified risks on overall project objectives (e.g., using Monte Carlo simulation to predict cost/schedule outcomes).

2.4.3. Response Strategies and Contingency Reserves

Strategies for Threats (Negative Risks) :

- **Avoid:** Eliminate the threat entirely.
- **Mitigate:** Reduce the probability and/or impact.

- **Transfer:** Shift ownership to a third party (e.g., insurance).
- **Accept:** Acknowledge the risk and potentially establish a **Contingency Reserve** (time or money) to handle it.
- **Escalate:** Move the risk up the hierarchy if it is outside the project's scope.

Strategies for Opportunities (Positive Risks):

- **Exploit:** Ensure the opportunity happens.
- **Share:** Allocate ownership to a third party best able to capture it.
- **Enhance:** Increase the probability/impact.
- **Accept:** Be willing to take advantage if it occurs.

2.4.4. The Risk Register

The **Risk Register** is the central document where all risk information is logged and tracked. It is a living document containing fields such as Description (with triggers), Probability, Impact, Response Strategy, Owner, and Status .

2.5. Project Initiation and Discovery

The success of a project is often determined before the first line of code is written. A structured initiation and discovery phase sets the trajectory for the entire engagement.

2.5.1. Discovery Phase and Objectives

The **Discovery Phase** is a mini-project in itself. Its goal is to define the scope, budget, resource plan, milestones, and assumptions of the overall project in a bounded timeframe. It is typically undertaken when a client needs a solution but cannot yet define the plan or estimate the cost.

+1

2.5.2. Execution of Discovery (Steps)

A successful discovery phase typically follows these steps :

1. **Gather Info:** Collect project details from Sales/Account Managers (budget, deadlines, constraints, staffing location restrictions).
2. **Analyze Client:** Understand decision-makers, business processes (change approval), scope, and enterprise architecture.
3. **Assemble Team:** Gather the discovery team.

4. **Plan:** Develop a high-level discovery plan.
5. **Access:** Get access to the client's infrastructure.
6. **Solutioning:** Define the solution to the client's problem.
7. **Proposal:** Estimate the solution, prepare the pitch, and present the Solution Proposal.

2.5.3. Initial Artifacts

During initiation, several key artifacts are created to structure the project :

- Backlog / Scope of work
- Stakeholder Register
- Risk Register
- Target Architecture
- Roadmap
- Onboarding guides

2.5.4. Solution Proposal and Blueprint

The **Solution Proposal** is the definitive output of the discovery phase. It transforms the concept into a concrete plan. It is often preceded by a **Blueprint**, a strategic document providing a "helicopter view" of the market opportunity and business case.

A complete Solution Proposal typically includes the following sections :

- **Executive Summary:** A summary for decision-makers.
- **Project Description / Business Case:** The justification for the project.
- **High-level Requirements:** Including Non-Functional Requirements (NFRs).
- **List of Deliverables:** Concrete items to be handed over (apps, docs, prototypes).
- **Baseline Architecture:** The "As-Is" landscape.
- **Target Architecture:** The "To-Be" future state.
- **Technology Stack:** Selected tools and platforms.
- **Human Efforts:** Detailed effort breakdown by roles and types.
- **Team Composition:** The staffing plan.

- **Risks:** Top risks and contingency reserves.
- **Assumptions:** Conditions that must be true for the plan to hold.
- **Dependencies and Constraints:** External factors affecting the project.
- **Test Strategy:** Approach to QA and testing.
- **Change Management Procedure:** How scope changes will be handled.
- **Timeline / Roadmap:** Visual schedule of phases and milestones.
- **Budget (Commercials):** Calculated based on rates, team structure, and reserves.
- **Stakeholders:** Key roles and responsibilities.
- **Advantages:** Why the client should work with your team (EPAM).
- **Case Studies:** Relevant examples of past success.
- **Backlog:** Optionally augmented with epics, features, and stories.